

Wcash Protocol Specification

Version 2026.1.0 [Mainnet]

w.cash

September 18, 2026

Abstract. Wcash is a cryptocurrency derived from the Zcash protocol. It shares Zcash’s transaction format, its shielded payment protocol, and its Equihash proof of work, and it runs as an independent block chain secured by auxiliary proof of work reused from Zcash. A Zcash block header whose coinbase commits to a Wcash block, and whose hash meets the Wcash target, proves work for that Wcash block. This specification defines the Wcash consensus protocol, including block identity, the auxiliary proof-of-work format and its validation, difficulty adjustment, the block subsidy, coinbase rules, and the genesis block. Protocol differences from Zcash are also explained.

Keywords: auxiliary proof of work, cryptocurrency, emission schedule, financial privacy, merged mining, proof of work.

Contents

1	Introduction	3
1.1	Caution	3
1.2	High-level Overview	3
2	Notation	4
3	Concepts	5
3.1	The Block Chain	5
3.2	Block Identifiers	5
3.3	Parent Blocks and Auxiliary Proof of Work	5
3.4	Transactions and Value Pools	5
3.5	Block Subsidy and Coinbase Transactions	6
3.6	Mainnet and Testnet	6
4	Abstract Protocol	6
4.1	Hash Functions	6
4.2	Merged Mining Scheme	6
4.3	Target Classification	7
4.4	Chain Value Pool Balances	7
5	Concrete Protocol	8
5.1	Integers, Byte Sequences, and Endianness	8
5.2	Constants	8
5.3	Block Identifier	8

5.4	Auxiliary Merkle Tree	8
5.5	Merged Mining Carrier	9
5.6	Encodings of Addresses	9
5.7	Network Identifiers	9
6	Network Upgrades	9
7	Consensus Changes from Zcash	10
7.1	Transaction Encoding and Consensus	10
7.2	Block Header Encoding and Consensus	10
7.3	Auxiliary Proof of Work	11
7.4	Difficulty Adjustment	12
7.5	Definition of Work	12
7.6	Calculating Block Subsidy	12
7.7	Coinbase Transactions	13
7.8	Genesis Block	13
8	Merged Mining Considerations	14
8.1	Template Construction and Submission	14
8.2	Proxies	14
8.3	Proof Size and Validation Cost	14
8.4	Double-Spending Analysis	14
8.5	Other Considerations	15
9	Acknowledgements	15
10	Change History	15
11	References	15

1 Introduction

Wcash is a cryptocurrency derived from the Zcash protocol [Zcash-Protocol], which is itself an implementation of the Decentralized Anonymous Payment scheme Zerocash [BCGGMTV2014]. Wcash keeps Zcash’s transparent payment scheme, its Ironwood shielded payment protocol, and its version 6 transaction format. It replaces the parts of Zcash that define an independent network: block identity, the genesis block, network identifiers, address encodings, consensus branch identifiers, the proof-of-work check, difficulty adjustment, and the block subsidy.

A Wcash block carries no Equihash solution of its own. It carries an *auxiliary proof of work*: a Zcash block header with a valid Equihash solution, together with Merkle paths showing that the header commits to the Wcash block. Any Zcash miner can therefore secure Wcash with the same work it performs for Zcash, and one solution can produce a valid block on both chains.

In this document, technical terms for concepts that play an important role in Wcash are written in slanted text. Italics are used for emphasis. The symbol § precedes section numbers in cross-references.

The key words MUST, MUST NOT, SHOULD, SHOULD NOT, RECOMMENDED, and MAY in this document are to be interpreted as described in [RFC-2119] when they appear in small capitals. They may also appear in lower case as plain English words, without their normative meaning.

Where this document is silent, the rules of [Zcash-Protocol] for the NU6.3 network upgrade apply, as modified by § 7 ‘Consensus Changes from Zcash’ on page 10. Rules inherited without change are generally not restated.

This specification is structured as follows:

- Notation: definitions of notation used throughout the document;
- Concepts: the principal abstractions needed to understand the protocol;
- Abstract Protocol: the merged-mining scheme and its security requirements;
- Concrete Protocol: how the functions and encodings of the abstract protocol are instantiated;
- Network Upgrades: the upgrades active on Wcash networks;
- Consensus Changes from Zcash: how Wcash differs from Zcash at the consensus layer;
- Merged Mining Considerations: non-normative guidance for pools, and an analysis of double spending.

1.1 Caution

Wcash security depends on consensus. A program that diverges from consensus weakens or loses the security of every user who relies on it, and the cause of the divergence does not matter: it could be a bug, an error in this document, or other software on the network behaving unexpectedly.

A specification of intended behaviour is nevertheless essential for security analysis, for understanding the protocol, and for independent implementations. Errors in this specification should be reported to the Wcash project through <https://w.cash>.

1.2 High-level Overview

The following overview gives a concise summary of the ideas behind the protocol for readers familiar with Zcash. It is not part of the normative specification.

All value in Wcash belongs to a *chain value pool*: the *transparent pool* or the *Ironwood pool*. Transparent value works as in Bitcoin and Zcash, with public addresses and amounts. Value in the Ironwood pool is carried by *notes*, whose commitments and nullifiers are revealed on the block chain while their values and recipients stay hidden, exactly as specified for Zcash. The older Zcash pools are not used.

Wcash blocks are found by *merged mining*. A mining pool builds a Zcash block whose coinbase ends with a 44-byte *carrier* committing to a Wcash *candidate block*. Each Equihash solution for that Zcash header is compared with two targets. If the header hash meets the Wcash target, the pool attaches an auxiliary proof

Each Wcash block pays its miner the whole block subsidy plus fees. The subsidy follows a time-scaled version of Monero’s smooth emission curve [vanSaberh2013], preceded by a slow start and followed by a permanent tail.

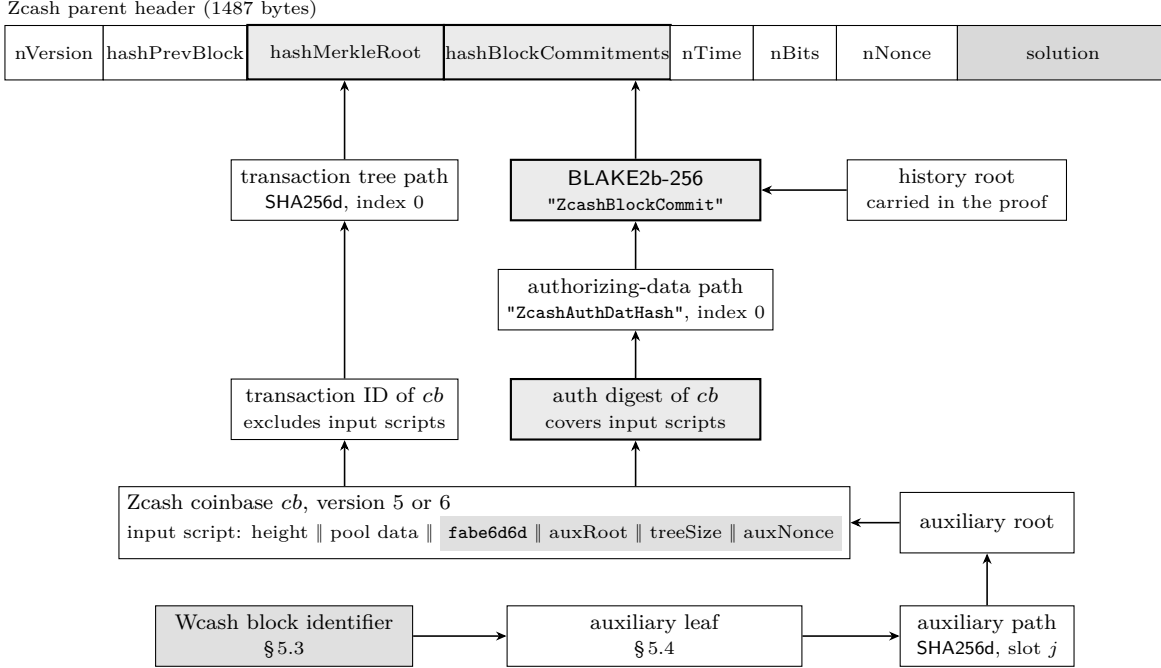


Figure 1. Authentication paths of an auxiliary proof of work. Arrows point from committed data toward the commitment that contains it.

2 Notation

$\mathbb{B}$ means the type of bit values, i.e. $\{0, 1\}$. $\mathbb{B}\mathbb{Y}$ means the type of byte values, i.e. $\{0 \dots 255\}$.

$\mathbb{N}$ means the type of nonnegative integers. $\mathbb{N}^+$ means the type of positive integers.

$x : T$ is used to specify that x has type T . A cartesian product type is denoted by $S \times T$, and a function type by $S \rightarrow T$.

$T^{[\ell]}$, where T is a type and ℓ is an integer, means the type of sequences of length ℓ with elements in T . $\mathbb{BY}^{[N]}$ means the type of byte sequences of arbitrary length.

$\text{length}(S)$ means the number of elements in S .

0x followed by a string of hexadecimal digits means the corresponding integer. $[0x00]^\ell$ means the sequence of ℓ zero bytes.

“...” means the given string represented as a sequence of bytes in US-ASCII.

$a \parallel b$ means the concatenation of sequences a then b .

$\text{le32}(x)$, for $x : \{0 \dots 2^{32} - 1\}$, means the 4-byte little-endian encoding of x .

$\text{rev}(S)$, for $S : \mathbb{BY}^{[32]}$, means the sequence S with its bytes in reverse order.

$|x|$ means the largest integer $\leq x$. $a \bmod q$ means the remainder on dividing a by q .

$\{a..b\}$ means the set of integers from a through b inclusive.

CompactSize refers to the variable-length integer encoding of Bitcoin and Zcash.

A 32-byte hash value is in *raw byte order* when it appears as the hash function outputs it, and in *display order* when its bytes are reversed and printed as hexadecimal, as RPC interfaces and block explorers do. Every hash value in this specification is in raw byte order unless stated otherwise.

3 Concepts

3.1 The Block Chain

At a given point in time, each full validator is aware of a set of candidate blocks. These form a tree rooted at the genesis block, where each node in the tree refers to its parent through the **hashPrevBlock** header field.

A path from the root toward the leaves of the tree consisting of one or more valid blocks is a *valid block chain*. The *block height* of the genesis block is 0, and the block height of each subsequent block increments by 1.

A node sums the work of all blocks in each valid block chain, as defined in § 7.5 ‘Definition of Work’ on page 12, and considers the valid block chain with the greatest total work to be best. Only Wcash work counts. Work performed for a parent block beyond the Wcash target contributes nothing.

3.2 Block Identifiers

Each Wcash block has a *block identifier* computed from its header with the auxiliary proof of work excluded, as specified in § 5.3 ‘Block Identifier’ on page 8. The block identifier is used in **hashPrevBlock**, in the auxiliary leaf that a parent block commits to, and throughout the peer-to-peer protocol.

Because the proof is excluded, two different auxiliary proofs of work can prove work for blocks with the same identifier. A node that needs to know which proof accompanies a block, such as a pool crediting its own work, **MUST** compare the proof itself as well as the identifier.

3.3 Parent Blocks and Auxiliary Proof of Work

A *parent block* is a Zcash block, version 5 or later, whose coinbase transaction commits to a Wcash block through a *carrier* (§ 5.5 ‘Merged Mining Carrier’ on page 9). Its header is the *parent header*. An *auxiliary proof of work* contains the parent coinbase, the parent header, and the Merkle paths that connect them and the carrier to the Wcash block identifier. Its encoding and validation are specified in § 7.3 ‘Auxiliary Proof of Work’ on page 11.

A parent block need not be accepted into the Zcash block chain. Wcash consensus depends only on the work the parent header represents.

3.4 Transactions and Value Pools

Each block contains one or more transactions, the first of which is a coinbase transaction. Every Wcash transaction is a version 6 transaction bound to the consensus branch identifier of its Wcash network (§ 6 ‘Network Upgrades’ on page 9).

Wcash has two chain value pools, the transparent pool and the Ironwood pool. The Sprout, Sapling, and Orchard pools of Zcash exist in the transaction format but are always empty on Wcash.

3.5 Block Subsidy and Coinbase Transactions

Wcash creates currency when blocks are mined. The value created on mining a block is the *block subsidy*, calculated as specified in § 7.6 ‘Calculating Block Subsidy’ on page 12. The entire block subsidy, together with the fees of the block’s transactions, is paid to the miner by the coinbase transaction. Wcash has no Founders’ Reward, funding streams, or lockbox.

3.6 Mainnet and Testnet

The production Wcash network, which supports the WEC token, is called *Mainnet*. There is also a public test network called *Testnet*, which supports a TWC token that is intended to have no monetary value. The Testnet block chain is subject to being reset at any time.

We call the smallest units of currency (on either network) *zatoshi*. On Mainnet, 1 WEC = 10^8 zatoshi. On Testnet, 1 TWC = 10^8 zatoshi.

Network-specific parameters are given in Table 1. Parameters marked as fixed at launch are determined by the release that activates the network and are published with it.

Table 1. Network-specific parameters.

Parameter	Mainnet	Testnet
Network identity label	“Wcash/mainnet/v1”	“Wcash/testnet/v7”
Peer-to-peer magic	0xc1e0dc1b	0x49d2934b
Consensus branch ID	fixed at launch	0x54ba2bfb
Genesis anchor source	Zcash Mainnet	Zcash Testnet
PoWLimit (compact)	fixed at launch	0x1e2fabe8
Minimum-difficulty rule	no	yes (§ 7.4)

4 Abstract Protocol

4.1 Hash Functions

$\text{SHA256d} : \mathbb{BY}^{[N]} \rightarrow \mathbb{BY}^{[32]}$ is SHA-256 applied twice. $\text{BLAKE2b-256}_P : \mathbb{BY}^{[N]} \rightarrow \mathbb{BY}^{[32]}$ is BLAKE2b with a 32-byte output and a 16-byte personalization P .

The hash function of the auxiliary Merkle tree is

$$\text{MerkleCRH}^{\text{Aux}} : \mathbb{BY}^{[32]} \times \mathbb{BY}^{[32]} \rightarrow \mathbb{BY}^{[32]}, \quad \text{MerkleCRH}^{\text{Aux}}(a, b) := \text{SHA256d}(a \parallel b).$$

Security requirement: SHA256d , $\text{MerkleCRH}^{\text{Aux}}$, and each personalized instance of BLAKE2b-256 used in this specification must be collision-resistant.

4.2 Merged Mining Scheme

A *merged mining scheme* for a child chain with chain identifier χ consists of three algorithms:

$$\begin{aligned} \text{AuxCommit} &: \mathbb{BY}^{[32]} \times \{0 \dots 2^{32} - 1\} \times \{0 \dots \text{MaxAuxTreeDepth}\} \rightarrow \mathbb{BY}^{[44]} \times \text{AuxPath} \\ \text{AuxProve} &: \text{ParentBlock} \times \text{AuxPath} \rightarrow \text{AuxPoW} \\ \text{AuxVerify} &: \mathbb{BY}^{[32]} \times \text{AuxPoW} \times \mathbb{N} \rightarrow \mathbb{B} \end{aligned}$$

$\text{AuxCommit}(\text{id}, n, d)$ returns a carrier for block identifier id with nonce n in an auxiliary tree of depth d , together with the path from its leaf to the auxiliary root. AuxProve takes a parent block whose coinbase ends

with that carrier and returns an auxiliary proof of work. $\text{AuxVerify}(\text{id}, \pi, T)$ returns 1 when π proves work at target T for id .

Their instantiations are specified in §5.4 ‘Auxiliary Merkle Tree’ on page 8, §5.5 ‘Merged Mining Carrier’ on page 9, and §7.3 ‘Auxiliary Proof of Work’ on page 11.

Security requirements:

- *Binding.* It must be infeasible to find a parent header H and auxiliary proofs of work π, π' containing H such that $\text{AuxVerify}(\text{id}, \pi, T) = \text{AuxVerify}(\text{id}', \pi', T') = 1$ and $\text{id} \neq \text{id}'$.
- *Domain separation.* A carrier created for chain identifier χ' must not yield an auxiliary proof of work that verifies for a chain with identifier $\chi \neq \chi'$.
- *Work soundness.* $\text{AuxVerify}(\text{id}, \pi, T) = 1$ must imply that π contains a parent header with a valid Equihash solution whose SHA256d hash, as an integer, is at most T .

Theorem 4.2.1 (Binding of auxiliary proofs of work). *If the hash functions of §4.1 ‘Hash Functions’ on page 6 are collision-resistant, the instantiation of §7.3 ‘Auxiliary Proof of Work’ on page 11 satisfies Binding.*

Proof. Let cb and cb' be the coinbases in π and π' . Both transaction paths start at index 0 and reach hashMerkleRoot of H , so distinct transaction identifiers would give a SHA256d collision. Both authorizing-data paths start at index 0 and yield roots that produce $\text{hashBlockCommitments}$ of H , so distinct authorizing digests would give a BLAKE2b collision. Together, the transaction identifier and the authorizing digest commit to the whole coinbase, including its input script [ZIP-244], so cb and cb' carry the same miner data. The carrier is the unique occurrence of the marker at the end of that data, so both proofs parse the same auxiliary root, tree size, and nonce, and therefore the same slot. Two distinct leaves at one slot under one root would give a $\text{MerkleCRH}^{\text{Aux}}$ collision, and equal leaves with the same prefix imply $\text{id} = \text{id}'$ unless SHA256d collides. $\square$

Non-normative note: Binding ensures that one Equihash solution cannot extend two competing Wcash blocks. Domain separation follows because the chain identifier enters every auxiliary leaf.

4.3 Target Classification

Let x be the SHA256d hash of a parent header interpreted as a 256-bit little-endian integer, let T_W be the Wcash target, and let T_Z be the Zcash target. The parent header is a *Wcash winner* if $x \leq T_W$ and a *Zcash winner* if $x \leq T_Z$.

Theorem 4.3.1 (Nested winners). *A parent header is both a Wcash winner and a Zcash winner if and only if $x \leq \min(T_W, T_Z)$. No parent header is a winner only on the chain with the smaller target.*

Proof. Both conditions compare the same integer x with a threshold, so the set of winners for the smaller threshold is contained in the set of winners for the larger. $\square$

Non-normative note: Let α be the fraction of Zcash’s hashrate that commits to Wcash. When both chains are at their target spacing of 75 seconds, $T_W/T_Z \approx 1/\alpha$, and Wcash difficulty settles at about α times Zcash difficulty. Every Zcash winner found by a participating pool is then also a Wcash winner.

4.4 Chain Value Pool Balances

Let $V^{\text{transparent}}(h)$ be the total value of unspent transparent outputs, and $V^{\text{Ironwood}}(h)$ the Ironwood pool balance, after the block at height h . Since every coinbase pays exactly the block subsidy plus fees, and fees transfer existing value,

$$\sum_{i=0}^h \text{BlockSubsidy}(i) = V^{\text{transparent}}(h) + V^{\text{Ironwood}}(h).$$

Both terms on the right are computable from public block chain data, so total issuance can be audited without decrypting any note, including notes created by private coinbase transactions (§ 7.7 ‘Coinbase Transactions’ on page 13).

5 Concrete Protocol

5.1 Integers, Byte Sequences, and Endianness

All integers in Wcash-specific encodings are little-endian. All 32-byte hash values are in raw byte order (§ 2 ‘Notation’ on page 4). The auxiliary root is the one exception: the carrier stores it byte-reversed, as $\text{rev}(\text{root})$.

5.2 Constants

Constant	Value
WcashChainId	0x57434153 (ASCII <code>WCAS</code>)
WcashHeaderVersion	0xd7430001
AuxLeafDomain	“Wcash/ZcashAuxPoW/leaf/v2” [0x00]
AuxMarker	[0xfa, 0xbe, 0x6d, 0x6d]
AuxProofMagic, AuxProofVersion	“WCAZ”, 2
MaxAuxProofSize	262144 bytes
MaxParentCoinbaseSize	131072 bytes
MaxParentBranchDepth	32
MaxAuxTreeDepth	16
PoWTargetSpacing	75 seconds
PoWAveragingWindow, PoWMedianBlockSpan, PoWDampingFactor	17, 11, 4
PoWMaxAdjustUp, PoWMaxAdjustDown	16%, 32%
SlowStartInterval	40000
EmissionParameter	$\lfloor (2^{64} - 1)/10^4 \rfloor = 1844674407370955$
TailSubsidy	37500000
COINBASE_MATURITY	100
MAX_BLOCK_SIZE	2000000

5.3 Block Identifier

Let the fields of a Wcash block header be as in § 7.2 ‘Block Header Encoding and Consensus’ on page 10. The block identifier is

$$\text{BlockId} := \text{BLAKE2b-256}_{\text{“WcashBlockIdV1”} \parallel [0x00]^2} (\text{le32}(\text{nVersion}) \parallel \text{hashPrevBlock} \parallel \text{hashMerkleRoot} \parallel \text{hashBlockCommitments} \parallel \text{le32}(\text{nTime}) \parallel \text{le32}(\text{nBits}) \parallel \text{nNonce}).$$

The fields `auxPowSize` and `auxPow` are not inputs.

5.4 Auxiliary Merkle Tree

The *auxiliary leaf* of a block identifier id is

$$\text{AuxLeaf}(\text{id}) := \text{SHA256d}(\text{AuxLeafDomain} \parallel \text{le32}(\text{WcashChainId}) \parallel \text{id}).$$

An auxiliary tree of depth $d : \{0 \dots \text{MaxAuxTreeDepth}\}$ has 2^d leaves, and each internal node is $\text{MerkleCRH}^{\text{Aux}}$ of its two children. For an auxiliary nonce $n : \{0 \dots 2^{32} - 1\}$, the Wcash leaf occupies slot

$$\text{AuxSlot}(n, d) := \left(\left(((n \cdot a + c + \text{WcashChainId}) \bmod 2^{32}) \cdot a + c \right) \bmod 2^{32} \right) \bmod 2^d,$$

where $a = 1103515245$ and $c = 12345$. Other merge-mined chains may occupy the remaining slots.

5.5 Merged Mining Carrier

The *carrier* is the 44-byte string of Table 2. It MUST form the final 44 bytes of the *miner data* of the parent coinbase, meaning the bytes of the coinbase input script that follow its canonical height encoding. Other data may precede it. The carrier is not wrapped in a script push opcode.

Table 2. Carrier encoding.

Bytes	Name	Data Type	Description
4	marker	char[4]	AuxMarker.
32	auxRoot	char[32]	rev of the root of the auxiliary tree.
4	treeSize	uint32	2^d , the number of leaves of the auxiliary tree.
4	auxNonce	uint32	The auxiliary nonce n .

5.6 Encodings of Addresses

Wcash uses the receiver formats of Zcash with Wcash-specific encodings. For Unified Addresses, the human-readable part is used both in the F4Jumble padding and in the Bech32m checksum [ZIP-316], so an address for one network or chain never parses as an address for another. Transparent addresses use Base58Check with Wcash-specific version bytes.

Address type	Mainnet prefix	Testnet prefix
Unified Address	wu1	wutest1
Transparent-source-only address [ZIP-320]	wtex1	wtextest1
Transparent P2PKH address	W1	WT
Transparent P2SH address	W3	WU
Sapling address (reserved, always invalid)	ws	wtestsapling

A Wcash node MUST reject Zcash addresses and addresses for a different Wcash network wherever it accepts an address.

5.7 Network Identifiers

The peer-to-peer magic of a Wcash network is the first four bytes of SHA-256 of its network identity label (Table 1). A reset network receives a new label version, and hence a new magic.

6 Network Upgrades

Wcash has no history before the NU6.3 network upgrade. On every Wcash network, the consensus rules of NU6.3 [Zcash-Protocol], as modified by §7 ‘Consensus Changes from Zcash’ on page 10, are active from block height 1. Earlier Zcash upgrades have no separate activation on Wcash.

Each Wcash network has its own consensus branch identifier (Table 1), distinct from every Zcash branch identifier. Any future Wcash network upgrade will define a new consensus branch identifier and an activation height.

7 Consensus Changes from Zcash

7.1 Transaction Encoding and Consensus

Consensus rules:

- Every transaction in a block other than the genesis block **MUST** be a version 6 transaction.
- The `nConsensusBranchId` field of every transaction **MUST** equal the consensus branch identifier of the Wcash network.
- A transaction **MUST NOT** contain Sprout, Sapling, or Orchard components. Shielded value is held only in the Ironwood pool.
- A transaction that spends a transparent output of a coinbase transaction **MUST NOT** be mined earlier than `COINBASE_MATURITY` blocks after that coinbase, and **MUST NOT** have any transparent outputs.

Non-normative note: The branch identifier enters transaction identifiers, signature hashes, and shielded authorization. A Wcash transaction is therefore invalid on Zcash, and a Zcash transaction is invalid on Wcash.

7.2 Block Header Encoding and Consensus

Table 3. Block header encoding.

Bytes	Name	Data Type	Description
4	<code>nVersion</code>	<code>uint32</code>	The block version number.
32	<code>hashPrevBlock</code>	<code>char[32]</code>	The block identifier of the previous block.
32	<code>hashMerkleRoot</code>	<code>char[32]</code>	The root of the transaction tree, as in Zcash.
32	<code>hashBlockCommitments</code>	<code>char[32]</code>	The block commitments, as in Zcash from NU5, computed over the Wcash chain history. For the genesis block, the anchor commitment of § 7.8.
4	<code>nTime</code>	<code>uint32</code>	The block time as a Unix timestamp.
4	<code>nBits</code>	<code>uint32</code>	The Wcash target in compact form.
32	<code>nNonce</code>	<code>char[32]</code>	An arbitrary field.
1, 3, or 5	<code>auxPowSize</code>	<code>compactSize</code>	The length of <code>auxPow</code> .
<code>auxPowSize</code>	<code>auxPow</code>	<code>char[]</code>	The auxiliary proof of work (§ 7.3).

Consensus rules:

- `nVersion` **MUST** be `WcashHeaderVersion`. This value has its high bit set, which Zcash forbids, so no Zcash header is a valid Wcash header.
- `auxPowSize` **MUST** be at most `MaxAuxProofSize`. `auxPow` **MUST** be empty for the genesis block and **MUST** be a valid auxiliary proof of work for every other block.
- `nBits` **MUST** equal `ThresholdBits(height)` as defined in § 7.4 ‘Difficulty Adjustment’ on page 12, subject to the Testnet minimum-difficulty rule.
- For each block other than the genesis block, `nTime` **MUST** be greater than `MedianTime(height)` and **MUST** be less than or equal to `MedianTime(height) + 90 · 60`.
- The size of a block **MUST** be at most `MAX_BLOCK_SIZE` bytes.

7.3 Auxiliary Proof of Work

Table 4. Auxiliary proof of work encoding.

Bytes	Name	Data Type	Description
4	magic	char[4]	AuxProofMagic.
1	version	uint8	AuxProofVersion.
varies	coinbaseSize	compactSize	The length of coinbase .
coinbaseSize	coinbase	char[]	The parent coinbase transaction.
varies	txBranchLen	compactSize	The depth of the transaction path.
32· txBranchLen	txBranch	char[32][]	The transaction path.
4	txIndex	uint32	The position of coinbase ; always 0.
varies	authBranchLen	compactSize	The depth of the authorizing-data path.
32· authBranchLen	authBranch	char[32][]	The authorizing-data path.
4	authIndex	uint32	The position of coinbase ; always 0.
32	historyRoot	char[32]	The chain history root of the parent block.
varies	auxBranchLen	compactSize	The depth d of the auxiliary tree.
32· auxBranchLen	auxBranch	char[32][]	The auxiliary path.
4	auxIndex	uint32	The slot of the Wcash leaf.
1487	parentHeader	char[1487]	The Zcash parent header, including its Equihash solution.

Let id be the block identifier of the Wcash block and $T_W = \text{ToTarget}(\text{nBits})$ its target.

Consensus rules:

1. **auxPow** MUST decode exactly as in Table 4 with no trailing bytes, using minimal CompactSize encodings. **magic** and **version** MUST have the values given.
2. **coinbaseSize** MUST be at most **MaxParentCoinbaseSize**. **txBranchLen** and **authBranchLen** MUST be equal and at most **MaxParentBranchDepth**. **auxBranchLen** MUST be at most **MaxAuxTreeDepth**. **txIndex** and **authIndex** MUST be 0.
3. **coinbase** MUST be a canonically encoded Zcash coinbase transaction of version 5 or 6.
4. The Merkle root computed from the transaction identifier of **coinbase** and from **txBranch** MUST equal the **hashMerkleRoot** field of **parentHeader**.
5. Let r_{auth} be the root computed from the authorizing-data digest of **coinbase** and **authBranch**. Then

$$\text{BLAKE2b-256}_{\text{"ZcashBlockCommit"}}(\text{historyRoot} \parallel r_{\text{auth}} \parallel [0x00]^{32})$$

MUST equal the **hashBlockCommitments** field of **parentHeader**.

6. **AuxMarker** MUST occur exactly once in the miner data of **coinbase**, as the first four bytes of its final 44 bytes, which are parsed as a carrier (§ 5.5). **treeSize** MUST equal $2^{\text{auxBranchLen}}$.
7. The value of **auxIndex** MUST equal **AuxSlot(auxNonce, auxBranchLen)**. The root computed from **AuxLeaf(id)**, **auxBranch**, and **auxIndex** MUST equal **rev(auxRoot)**.
8. $\text{SHA256d}(\text{parentHeader})$, interpreted as a 256-bit little-endian integer, MUST be at most T_W .
9. The Equihash solution in **parentHeader** MUST be valid with parameters $n = 200$, $k = 9$ and personalization “ZcashPoW”, as in [Zcash-Protocol].

Non-normative notes:

- The **nBits** field of **parentHeader** is not used. Only the Wcash target, which Wcash headers alone determine, decides validity.
- The transaction identifier of a version 5 or 6 transaction excludes transparent input scripts [ZIP-244]. Rule 5 is what authenticates the carrier.
- A parent pool gives up 44 of the 100 bytes that Zcash permits for a coinbase input script.

7.4 Difficulty Adjustment

Wcash uses the difficulty adjustment algorithm of [Zcash-Protocol, § 7.7.3] with the constants of § 5.2 ‘Constants’ on page 8, applied to Wcash block headers only. With `ToTarget` and `ToCompact` as in [Zcash-Protocol], and h the height of the block:

$$\begin{aligned}
\text{AveragingWindowTimespan} &:= \text{PoWAveragingWindow} \cdot \text{PoWTargetSpacing} \\
\text{MinActualTimespan} &:= \lfloor \text{AveragingWindowTimespan} \cdot (1 - \text{PoWMaxAdjustUp}) \rfloor \\
\text{MaxActualTimespan} &:= \lfloor \text{AveragingWindowTimespan} \cdot (1 + \text{PoWMaxAdjustDown}) \rfloor \\
\text{MedianTime}(h) &:= \text{median}(\lfloor \text{nTime}(i) \text{ for } i \text{ from } \max(0, h - \text{PoWMedianBlockSpan}) \\
&\quad \text{up to } h - 1 \rfloor) \\
\text{ActualTimespan}(h) &:= \text{MedianTime}(h) - \text{MedianTime}(h - \text{PoWAveragingWindow}) \\
\text{ActualTimespanDamped}(h) &:= \text{AveragingWindowTimespan} + \frac{\text{ActualTimespan}(h) - \text{AveragingWindowTimespan}}{\text{PoWDampingFactor}} \\
\text{ActualTimespanBounded}(h) &:= \max(\text{MinActualTimespan}, \\
&\quad \min(\text{MaxActualTimespan}, \text{ActualTimespanDamped}(h))) \\
\text{MeanTarget}(h) &:= \left\lfloor \sum_{i=1}^{\text{PoWAveragingWindow}} \text{ToTarget}(\text{nBits}(h - i)) / \text{PoWAveragingWindow} \right\rfloor \\
\text{Threshold}(h) &:= \min\left(\text{PoWLimit}, \left\lfloor \frac{\text{MeanTarget}(h)}{\text{AveragingWindowTimespan}} \right\rfloor \cdot \text{ActualTimespanBounded}(h)\right) \\
\text{ThresholdBits}(h) &:= \text{ToCompact}(\text{Threshold}(h))
\end{aligned}$$

Heights with fewer than `PoWAveragingWindow` predecessors are handled as in [Zcash-Protocol, § 7.7.3].

Consensus rule: [Testnet only] If `nTime` of a block is more than $6 \cdot \text{PoWTargetSpacing}$ seconds after `nTime` of the preceding block, its `nBits` MAY be `ToCompact(PoWLimit)`.

7.5 Definition of Work

The work of a block is $\lfloor 2^{256} / (\text{ToTarget}(\text{nBits}) + 1) \rfloor$, using the `nBits` of the Wcash block header. The work represented by the parent header beyond the Wcash target is not counted.

7.6 Calculating Block Subsidy

All amounts are in zatoshi, and every division rounds toward zero. Let $M := \text{EmissionParameter}$, $T := \text{TailSubsidy}$, and $R_0 := \lfloor 5M/2^{22} \rfloor = 2199023255$. For $\text{height} > \text{SlowStartInterval}$, let $E(\text{height})$ be the sum of $\text{BlockSubsidy}(i)$ for i from $\text{SlowStartInterval} + 1$ up to $\text{height} - 1$. Then

$$\text{BlockSubsidy}(\text{height}) := \begin{cases} 0, & \text{if } \text{height} = 0, \\ \lfloor R_0 \cdot \text{height} / \text{SlowStartInterval} \rfloor, & \text{if } 1 \leq \text{height} \leq \text{SlowStartInterval}, \\ \max(\lfloor 5 \cdot (M - E(\text{height})) / 2^{22} \rfloor, T), & \text{otherwise.} \end{cases}$$

$\text{MinerSubsidy}(\text{height}) := \text{BlockSubsidy}(\text{height})$. Wcash has no funding streams, lockbox disbursements, or Founders’ Reward.

Non-normative notes:

- The schedule is a time-scaled adaptation of Monero’s emission rule. Monero pays $(M - A)/2^{19}$ per 120-second block. A 75-second Wcash block pays five eighths of that, which matches Monero’s nominal emission per minute, and T is 0.375 WEC per block, equivalent to Monero’s 0.6 per 120 seconds. The schedule does not replay Monero’s historical issuance.

- The slow start raises the subsidy linearly to R_0 , 21.99023255 WEC, at height 40000, about 34.72 days after genesis. The curve begins at height 40001 with the same subsidy, and its first decrease occurs at height 40002. Slow-start issuance, about 439816 WEC, counts toward total issuance and is not included in E .
- M is an emission parameter, not a maximum supply. Slow-start issuance is additional, and the tail continues indefinitely, so there is no finite supply cap.
- The subsidy first equals T at height 3455360, approximately 8.22 years after genesis, when about 18571987 WEC have been issued. The tail then issues 157680 WEC per year.

7.7 Coinbase Transactions

Consensus rules:

- The total value of the outputs of the coinbase transaction MUST equal $\text{BlockSubsidy}(\text{height})$ plus the fees of all other transactions in the block, and MUST be positive.
- The outputs of a coinbase transaction MUST be either transparent outputs paying Wcash transparent addresses, or Ironwood outputs, and MUST NOT mix the two.
- An Ironwood output of a coinbase transaction MUST NOT be decryptable using the all-zero outgoing viewing key. This replaces the corresponding rule of [ZIP-213].
- A coinbase transaction MUST NOT contain funding stream or lockbox outputs.

Non-normative note: In the private mode, the recipient of an Ironwood coinbase output is hidden, while the value entering the Ironwood pool remains public through its value balance (§4.4).

7.8 Genesis Block

The genesis block of each Wcash network commits to one block of an external chain, the *anchor block*. For Mainnet, the anchor block is a Zcash Mainnet block. The *anchor* is encoded as in Table 5.

Table 5. Genesis anchor encoding.

Bytes	Name	Data Type	Description
1	version	uint8	1.
1	network	uint8	0 for Mainnet, 1 for Testnet.
1	source	uint8	1 for Zcash Testnet, 2 for Zcash Mainnet.
1	reserved	uint8	0.
4	height	uint32	The block height of the anchor block.
32	blockHash	char[32]	The hash of the anchor block in raw byte order.

Consensus rules:

- **hashBlockCommitments** of the genesis block MUST equal $\text{BLAKE2b-256}_{\text{"WcashZecAnchorV1"}}(\text{anchor})$.
- The genesis block MUST contain a single coinbase transaction with a single transparent output of value 0. Its input script contains the genesis statement

18/Sep/2026 Wcash (WEC) ZEC #<height> <hash>

where <height> is the decimal height and <hash> the display-order hash of the anchor block.

- **hashPrevBlock** and **nNonce** of the genesis block MUST be $[0x00]^{32}$, and **auxPow** MUST be empty.

Non-normative notes:

- The hash of the anchor block could not be known before that block was mined, so the genesis block cannot predate it, in the manner of the newspaper headline in Bitcoin’s genesis block [Nakamoto2008].

- The anchor is fixed only after the anchor block is deeply confirmed on its chain, and no node selects or replaces the anchor at runtime.
- The supply at genesis is zero.

8 Merged Mining Considerations

‘The system is secure as long as honest nodes collectively control more CPU power than any cooperating group of attacker nodes.’

Satoshi Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System” [Nakamoto2008]

This section is not part of the normative specification.

8.1 Template Construction and Submission

A participating pool runs a Wcash node, a Zcash node that builds its templates, and at least one independent Zcash node that checks them.

1. The Wcash node assembles a candidate block and provides its identifier, target, and serialized form.
2. The pool’s Zcash node builds an ordinary template, reserves 44 bytes of coinbase space, splices the carrier into the coinbase miner data, and recomputes the authorizing-data digest, the authorizing-data root, and `hashBlockCommitments`.
3. Independent Zcash nodes validate the exact parent block on the same tip before any miner receives it.
4. Miners receive the job over the Zcash Stratum protocol [ZIP-301]. The fifth field of the job carries the parent’s block commitments, which miners hash exactly as given.
5. A Wcash winner is packaged as an auxiliary proof of work and submitted to Wcash. A Zcash winner is submitted to Zcash as the validated block with the new nonce and solution. A failure on one chain does not suppress the other.

For hardware that evaluates ρ Equihash solutions per second, a share target of about $\lfloor 2^{256}/(\rho I) \rfloor$ yields one share every I seconds on average. A pool SHOULD advertise a share target no harder than the easier of the two network targets, so that no winner on either chain is lost.

8.2 Proxies

A pool cannot take part in Wcash through a proxy placed in front of another pool. Adding the carrier changes the coinbase’s authorizing data, hence `hashBlockCommitments` and the header that miners hash, and shares for that header are not shares for the upstream job. Participation requires control of template construction.

8.3 Proof Size and Validation Cost

For a parent coinbase of fewer than 253 bytes, a parent transaction tree of depth d , and an auxiliary tree of depth d_{aux} , an auxiliary proof of work occupies $1540 + \text{coinbaseSize} + 64d + 32d_{\text{aux}}$ bytes, about 2 to 2.5 kB for typical Zcash blocks. Validation costs one Equihash verification and $O(d + d_{\text{aux}})$ hash evaluations.

8.4 Double-Spending Analysis

An attacker who wants to reverse a payment must build an alternate Wcash chain faster than the honest chain. Let α be the share of Zcash’s hashrate honestly committed to Wcash, and h the attacker’s Equihash hashrate on the same scale. The honest miners and the attacker find the next Wcash block with probabilities

$$p = \frac{\alpha}{\alpha + h}, \quad q = \frac{h}{\alpha + h}.$$

As in [Nakamoto2008], the race is a random walk [Feller1957]: an attacker z blocks behind eventually catches up with probability $(q/p)^z$ when $q < p$, and with certainty otherwise. A recipient who waits z blocks gives the attacker Poisson-distributed progress with mean $\lambda = zq/p$, so the attack succeeds with probability

$$P = 1 - \sum_{k=0}^z \frac{\lambda^k e^{-\lambda}}{k!} \left(1 - (q/p)^{z-k}\right).$$

The following table gives the smallest z with $P < 0.1\%$. A dash marks an attacker with at least half of the committed work.

$\alpha \setminus h$	0.5%	1%	2%	5%	10%
0.05	5	8	21	–	–
0.10	4	5	8	34	–
0.25	3	3	5	8	21
0.50	2	3	3	5	8

8.5 Other Considerations

Merged mining lowers an attacker’s opportunity cost, because hardware that builds a private Wcash chain can keep submitting Zcash winners from the same work. Pools can also start or stop committing to Wcash without changing their hashing cost, so Wcash hashrate can change abruptly, and the damped retarget needs several averaging windows to follow a large change [JZSVW2017]. Recipients of large payments SHOULD wait for more confirmations while the committed share α is small.

Shielding is optional. Transparent payments, and value moving between the transparent and Ironwood pools, are visible on the block chain, and metadata such as timing can link transactions.

9 Acknowledgements

Wcash builds on the Zcash protocol and on the Zcash Protocol Specification [Zcash-Protocol], whose structure and conventions this document follows. Its merged-mining design follows the auxiliary proof of work introduced by Namecoin [MergedMining], and its emission schedule adapts the design of CryptoNote and Monero [vanSaberh2013].

10 Change History

2026.1.0 (2026-09-18)

- Initial version, specifying Wcash Mainnet.

11 References

- [BCGGMTV2014] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. *Zerocash: Decentralized Anonymous Payments from Bitcoin*. IEEE Symposium on Security and Privacy, 2014.
- [BK2016] Alex Biryukov and Dmitry Khovratovich. *Equihash: Asymmetric Proof-of-Work Based on the Generalized Birthday Problem*. Network and Distributed System Security Symposium, 2016.
- [Feller1957] William Feller. *An Introduction to Probability Theory and Its Applications*. 1957.

- [JZSVW2017] Aljosha Judmayer, Alexei Zamyatin, Nicholas Stifter, Artemios G. Voyiatzis, and Edgar Weippl. *Merged Mining: Curse or Cure?* Data Privacy Management, Cryptocurrencies and Blockchain Technology, 2017.
- [MergedMining] *Merged mining specification*. Bitcoin Wiki. https://en.bitcoin.it/wiki/Merged_mining_specification
- [Nakamoto2008] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008.
- [RFC-2119] Scott Bradner. *Key words for use in RFCs to Indicate Requirement Levels*. RFC 2119, 1997.
- [vanSaberh2013] Nicolas van Saberhagen. *CryptoNote v 2.0*. 2013.
- [Zcash-Protocol] Daira-Emma Hopwood, Sean Bowe, Taylor Hornby, and Nathan Wilcox. *Zcash Protocol Specification*. <https://zips.z.cash/protocol/protocol.pdf>
- [ZIP-213] *Shielded Coinbase*. Zcash Improvement Proposal 213.
- [ZIP-244] *Transaction Identifier Non-Malleability*. Zcash Improvement Proposal 244.
- [ZIP-301] *Zcash Stratum Protocol*. Zcash Improvement Proposal 301.
- [ZIP-316] *Unified Addresses and Unified Viewing Keys*. Zcash Improvement Proposal 316.
- [ZIP-320] *Defining an Address Type to which funds can only be sent from Transparent Addresses*. Zcash Improvement Proposal 320.